RETROSPECTIVA CICLO 4

PROYECTO DOPO — MoralesS

1. ¿Cuáles fueron los mini-ciclos definidos? Justíficalos.

Se definieron 8 mini-ciclos para el Ciclo 4 (Refactoring y Extensión). El orden separa primero la refactorización, que no cambia el comportamiento, de la extensión funcional, y cada mini-ciclo termina con un criterio verificable.

- **Mini-ciclo 1: Refactorización base.** Los símbolos dejan de ser textos (List<String>) y pasan a ser objetos Symbol; Wheel pasa a ser una clase abstracta con NormalWheel como la rueda de siempre. Es el cimiento de la extensibilidad: sin esto no se pueden agregar tipos. Criterio: las 65 pruebas de los ciclos 2 y 3 siguen pasando sin modificarlas.
- **Mini-ciclo 2: Ruedas por tipo.** addWheel(type, pos), una fábrica privada (createWheel) y la consulta wheelTypes(). Depende del 1, porque necesita las subclases.
- **Mini-ciclo 3: Rueda rebel.** Se agregan canBeLocked, canBeSwapped y canBeDeleted y se controlan en lock, swap y delWheel. Se hace antes que la lefty porque solo modifica operaciones de gestión de ruedas ya existentes.
- **Mini-ciclo 4: Rueda lefty.** Se agregan los ganchos turn y rotate(..., left) y el método leftOf. Es el único tipo que depende de una rueda vecina, por eso va después de tener el mecanismo de tipos estable.
- **Mini-ciclo 5: Símbolos por tipo.** addSymbol(type, pos, color), EphemeralSymbol, ShySymbol y el gancho select() que la máquina invoca cuando una rueda queda mostrando un símbolo.
- **Mini-ciclo 6: Distinción visual.** Requisito de usabilidad: una franja de color bajo cada rueda según su tipo, una marca en la esquina según el tipo de símbolo, forma cuadrada para el tímido y tamaño variable para el efímero. Va después de los símbolos porque necesita saber cómo se ve cada uno.
- **Mini-ciclo 7: Nuevo tipo propuesto (requisito 19) y ajuste del Contest.** Se agregó la rueda MirrorWheel, que gira en sentido contrario, como prueba de que agregar un tipo es crear una clase y una línea en la fábrica; y se aplicó MAX_RUEDAS = 8 en solve y simulate.
- **Mini-ciclo 8: Pruebas, aceptación y cierre.** SlotMachineC4Test, SlotMachineCC4Test, las dos pruebas de aceptación, el diseño en Astah, esta retrospectiva y la publicación en Git.

Justificación general: el orden respeta las dependencias reales del código (tipos antes de comportamientos, comportamientos antes de la parte visual) y sigue la práctica XP de entregas pequeñas y verificables, usando las pruebas anteriores como red de seguridad de la refactorización.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

De los 8 mini-ciclos, los mini-ciclos 1 a 8 están completos, implementados y funcionando en las clases SlotMachine, SlotMachineContest, Symbol (con NormalSymbol, EphemeralSymbol, ShySymbol) y Wheel (con NormalWheel, LeftyWheel, RebelWheel, MirrorWheel)

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

César Morales: 25 horas.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

El mayor logro fue lograr que la máquina fuera realmente extensible: pasamos de una sola clase de rueda y símbolos como textos a una jerarquía en la que SlotMachine ya no pregunta de qué tipo es cada elemento, sino que le pide que haga lo suyo. La prueba de que funcionó es que el último tipo que agregue (la rueda mirror) fue una clase pequeña que redefine un solo método y una línea en la fábrica, sin tocar ningún otro método. Además, al hacer primero la refactorización con las pruebas anteriores como red de seguridad, ninguna de las 65 pruebas de los ciclos 2 y 3 tuvo que modificarse.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue que los tipos se pudieran distinguir claramente en pantalla (requisito de usabilidad) con solo dos figuras disponibles, Rectangle y Circle, y un Canvas que dibuja en el orden en que se hacen visibles. Cada Wheel tenía un rectángulo y un círculo; lo resolvi dándole cinco figuras: una franja de color debajo de la rueda (identifica el tipo de rueda), un círculo y un cuadrado para el símbolo (el tímido es cuadrado) y un punto de marca en la esquina (identifica el tipo de símbolo, y sigue visible aunque el tímido esté oculto).

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

En este ciclo trabajé de forma individual, así que respondo desde lo que hice yo. Planifiqué los mini-ciclos de acuerdo con las dependencias reales del código y empecé por la refactorización, usando las pruebas de los ciclos 2 y 3 como red de seguridad antes de agregar cualquier tipo. También comprobé que las pruebas realmente detectan errores, rompiendo a propósito cada comportamiento nuevo para ver que fallaran.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿por qué?

La más útil fue el refactoring apoyado en pruebas automatizadas. Pude cambiar la representación de los símbolos y volver abstracta a Wheel con la certeza de que el comportamiento anterior seguía intacto, porque las 65 pruebas de los ciclos 2 y 3 seguían pasando. Al trabajar solo, esa red de seguridad fue todavía más importante, porque las pruebas cumplían el papel de segundo par de ojos. También ayudó el diseño simple: cada tipo nuevo es una clase pequeña con un único motivo para existir.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Oracle. (s.f.). *Class Random*. Java Platform SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Random.html
- Bechtold, S., Brannen, S., Link, J., Merdes, M., Philipp, M., de Rancourt, J., & Stein, C. (s.f.). *JUnit 5 User Guide*. https://junit.org/junit5/docs/current/user-guide/
- Anthropic. (2026). *Claude* [Modelo de lenguaje]. https://claude.ai